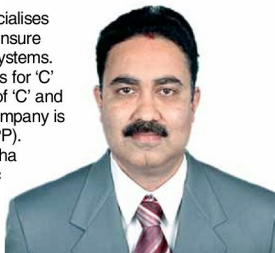


“Relying on manual code inspection could lead to fatal consequences”

Founded in 1985 and headquartered in the UK, PRQA specialises in defect prevention and promoting safe code practices to ensure reliability of safety-critical, mission-critical and commercial systems. It maintains full voting representation on ISO working groups for ‘C’ and ‘C++,’ and is a founding member of MISRA (producer of ‘C’ and ‘C++’ standards) committee. The programming research company is also originator of high-integrity ‘C++’ coding standard (HICPP).

Sanjay Shanbhag, director-India, spoke to Abhishek Mutha and Ashwin Gopinath of EFY about the importance of static analysis and why automated solutions be preferred over manual inspection methods

SANJAY SHANBHAG
DIRECTOR-INDIA, PRQA



What are your views on manual code inspection?

We advocate the use of automated code inspection process simply due to the exponential growth in the amount of code being generated and its complexity. For example, a simple washing machine now has one million lines of embedded code and it would be a gargantuan task for a human being to detect issues manually in this maze of code.

Another factor is that every time a fresh code is added to the existing code base, the entire code has to be inspected all over again, wasting time and effort when this can be automated. We, however, recognise that for the proper functioning of this automated process, human intervention is still necessary to control deviations and certain exception defects.

Are manual methods of coding inspection and defect detection still prevailing in India?

Yes, some companies still use manual methods. The trouble is that they are worried about the cost of tool investment, rather than looking at the long-term benefits of automating the process.

What is the difference between bug catching and coding standard compliance?

Basically, bug catching is finding defects, whereas coding standard compliance means adhering to best practices and therefore preventing introduction of defects in the first instance.

There are some static tools in the market that only look at finding defects and don't aid in process improvement.

We, on the other hand, enable our clients to improve their coding process and develop high-quality code by recommending tracking issues early and often during the development phase. We believe in the saying “prevention is better than cure.” Hence we enforce a coding standard to help our customers develop robust and high-integrity code.

The fundamental purposes of a coding standard is to define a safer sub-set of the programming language by framing a set of rules that eliminate coding constructs known to be hazardous, educating all developers to the same standard and promoting the best practice.

Are there any language insecurities while coding?

No coding language is perfect and all come with their advantages (such as flexibility) and flaws (known as undefined behaviour). Undefined behaviour occurs when language definition is unclear and no outcome specification has been defined, or where the compiler or library vendor has some explicit freedom of implementation.

Typically, ‘C’ language is used in embedded and safety/mission-critical applications (high cost-of-failure areas such as aerospace, military, automotive and medical devices). ‘C++,’ on the other hand, could be used mainly in human-machine interface. Its use has started in embedded as well as commercial applications.

How does static analysis tool assist developers in improving the quality of the code?

Static analysis is a crucial element of high-quality